Введение в Deep Learning

Основные концепции, проблемы и методы их решения

Курс «Deep Learning»

2026

План лекции

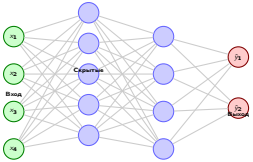
- **Что такое Deep Learning?**
- Нейронная сеть — базовые идеи
- Анатомия обучения: функция потерь, градиентный спуск
- **Проблемы глубоких сетей**
- **Методы решения:**
  - Продвинутые оптимизаторы
  - Инициализация весов
  - Функции активации
  - Нормализация
  - Регуляризация

Что такое Deep Learning?

Глубокое обучение — подраздел машинного обучения, использующий многослойные нейронные сети для извлечения иерархических признаков.

Ключевые идеи:

- **Сквозное обучение** (end-to-end): сырые данные $\rightarrow$ ответ
- **Иерархия признаков**: от простых к сложным
- **Масштабирование**: чем больше данных и параметров — тем лучше



Анатомия обучения: градиентный спуск

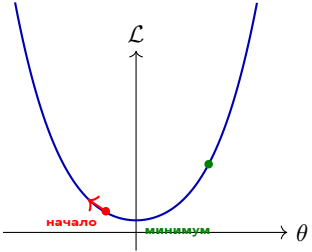
Стандартный цикл обучения

1. Forward pass: $y = f(x; \theta)$ 2. Loss $\mathcal{L}(y, y_{\text{true}})$ 3. Backward: $\nabla_\theta \mathcal{L}$ 4. Update: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$

Стохастический градиентный спуск (SGD):

$$\theta_{t+1} = \theta_t - \eta \, \nabla_\theta \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})$$

- η — learning rate
- Случайный мини-батч
- Шумный, но быстрый



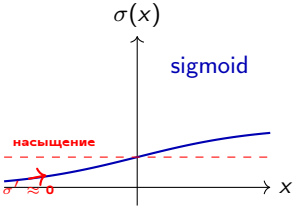
Проблема 1: Исчезающие и взрывающиеся градиенты

Причина: цепное правило при обратном распространении

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{k=2}^{L} \frac{\partial h_k}{\partial h_{k-1}}$$

Если $\|\partial h_k / \partial h_{k-1}\| < 1 \to$ **исчезает** (ранние слои не обучаются)
Если $> 1 \to$ **взрывается** (NaN, нестабильность)

Классический пример: sigmoid/tanh в глубоких сетях.



Производная sigmoid:
$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$
В 10 слоях: $0.25^{10} \approx 10^{-6}$ — градиент пропал.

Проблема 2: Переобучение (Overfitting)

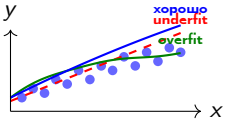
Модель запоминает шум, а не закономерности.

Симптомы:

- Loss на train $\to 0$
- Loss на val растёт
- Высокая variance

Факторы риска:

- Мало данных
- Слишком много параметров
- Шум в разметке



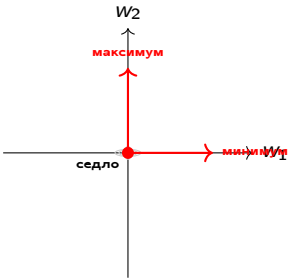
Проблема 3: Локальные минимумы и седловые точки

В многомерных пространствах седловые точки встречаются **гораздо чаще**, чем локальные минимумы.

Почему это проблема?

В седловой точке $\nabla f = 0$, но в одних направлениях минимум, в других — максимум. SGD «застревает».

Современное понимание: большинство локальных минимумов в DL **близки** к глобальному (Loss landscape).



$f(w_1, w_2) = w_1^2 - w_2^2$ — седловая точка в $(0, 0)$

Проблема 4: Вычислительная сложность и ковариационный сдвиг

Вычислительные затраты:

- Один проход по данным может стоить дни/недели
- Подбор гиперпараметров — экспоненциальный поиск
- Память: модель + градиенты + оптимизатор
- GPU — почти необходимость

Internal Covariate Shift:

Распределение входов каждого слоя постоянно меняется, потому что меняются веса предыдущих слоёв.

Последствия:

- Приходится снижать learning rate
- Осторожная инициализация
- Зависимость от порядка данных

Решение 1: Продвинутые оптимизаторы

SGD с Momentum:

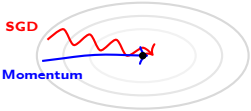
$$v_t = \gamma v_{t-1} + \eta \nabla \mathcal{L}(\theta_t)$$
$$\theta_{t+1} = \theta_t - v_t$$

«Шар, катящийся с горы» — накапливает инерцию.

Adam (Kingma & Ba, 2015):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$
$$\hat{\theta}_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \varepsilon}$$

Momentum + адаптивный learning rate для каждого параметра.



Оптимизатор	Скорость	Качество
SGD	медленно	хорошо
Momentum	средне	хорошо
Adam	быстро	отлично
AdamW	быстро	отлично + L2

Решение 2: Функции активации

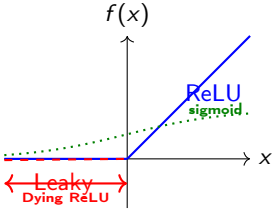
ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

- Нет насыщения для $x > 0 \to$ не убивает градиент
- Вычислительно дёшево
- **Проблема:** «умирающий ReLU» (нейроны залипают на 0)

Варианты:

- **Leaky ReLU:** $f(x) = \max(\alpha x, x)$, $\alpha \approx 0.01$
- **ELU:** плавное затухание для $x < 0$
- **Swish/SiLU:** $f(x) = x \cdot \sigma(x)$
- **GELU:** $f(x) = x \cdot \Phi(x)$ (BERT, GPT)



	Градиент	Симметрия
Sigmoid	$\to 0$ при $\lvert x\rvert > 2$	да
Tanh	$\to 0$ при $\lvert x\rvert > 3$	да
ReLU	0 или 1	нет
Swish	плавный	нет

Решение 3: Инициализация весов

Проблема

Если веса слишком большие — градиенты взрываются. Если слишком маленькие — градиенты исчезают. Нужна «золотая середина»: дисперсия сигнала сохраняется между слоями.

Xavier/Glorot (2010):

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

Подходит для sigmoid, tanh.

He/Kaiming (2015):

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

Оптимальна для ReLU и его вариантов.

Интуиция:

Рассмотрим слой: $y = \sum_{i=1}^{n} w_i x_i$

Если $\text{Var}(w) = \sigma^2$ и $\text{Var}(x) = 1$:

$$\text{Var}(y) = n\sigma^2$$

Чтобы $\text{Var}(y) = \text{Var}(x)$:

$$\sigma^2 = \frac{1}{n}$$

Аналогично для обратного прохода $\rightarrow \frac{2}{n_{\text{in}}}$ для ReLU.

Решение 4: Batch Normalization (Ioffe & Szegedy, 2015)

Идея: нормализуем активации каждого слоя к нулевому среднему и единичной дисперсии.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_{\text{batch}}^{(k)}}{\sqrt{\sigma_{\text{batch}}^{2(k)} + \varepsilon}}$$

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

Эффекты:

- Позволяет использовать **высокий learning rate**
- Снижает зависимость от инициализации
- Действует как **регуляризатор** (шум от мини-батча)
- Ускоряет сходимость в 10+ раз

На инференсе: используются скользящие средние μ и σ, накопленные во время обучения.

Альтернативы:

- **LayerNorm** — по признакам (Transformer)
- **InstanceNorm** — для стилей
- **GroupNorm** — для маленьких батчей

Key insight: нормализация делает loss landscape более гладким $\rightarrow$ градиенты более предсказуемы.

Решение 5: Регуляризация

L1/L2 Регуляризация (Weight Decay):

$$\tilde{\mathcal{L}} = \mathcal{L} + \lambda \|\theta\|_2^2$$

Штраф за большие веса. L2 = Gaussian prior, L1 = Laplace prior.

Dropout (Srivastava et al., 2014):

$$h_{\text{dropped}} = h \odot m, \quad m_i \sim \text{Bernoulli}(p)$$

На каждом проходе случайно «выключаем» долю p нейронов.

Ensemble-эффект: модель учится работать без каждой отдельной фичи.

Data Augmentation:

- Искусственное расширение датасета
- Для изображений: повороты, сдвиги, цвета
- Для текста: back-translation, word dropout
- Один из самых эффективных приёмов

Early Stopping:

- Следим за validation loss
- Останавливаемся, когда val loss перестаёт уменьшаться
- Сохраняем лучшую модель

Label Smoothing:

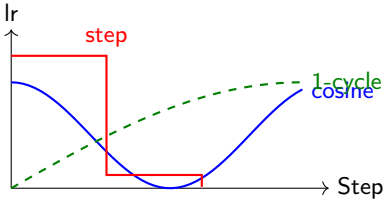
$$y_{\text{smooth}} = (1 - \varepsilon) y_{\text{one-hot}} + \varepsilon / K$$

Решение 6: Планирование learning rate

Learning rate — самый важный гиперпараметр. Его нужно уменьшать по ходу обучения.

Стратегии:

- **Step decay:** уменьшаем в 10x каждые N эпох
- **Cosine annealing:** плавное уменьшение по косинусу
- **Cosine warmup:** линейный рост $\to$ косинус
- **Reduce on plateau:** уменьшаем, когда val loss застыл
- **One-cycle:** возрастание $\to$ убывание



Warmup (первые шаги):
Если начать с большого lr сразу — градиенты «улетят». Начинаем с 0 и линейно растём до базового lr.

Сводка: проблемы и решения

Проблема		Решение
Исчезающий градиент	$\longrightarrow$	ReLU, He init, BatchNorm, ResNet
Взрывающийся градиент	$\longrightarrow$	Gradient clipping, BatchNorm
Переобучение	$\longrightarrow$	Dropout, Weight Decay, Augmentation
Медленная сходимость	$\longrightarrow$	Adam, Momentum, LR schedule
Внутренний ковар. сдвиг	$\longrightarrow$	Batch/Layer Normalization
Плохая инициализация	$\longrightarrow$	He/Xavier init
Седловые точки	$\longrightarrow$	Momentum, Adam (инерция)

Главный урок

Глубокие сети работают не благодаря одному приёму, а благодаря **комбинации**: правильная инициализация + нормализация + продвинутый оптимизатор + регуляризация + планирование lr.

Заключение

Что мы узнали:

- Deep Learning = иерархическое представление
- Градиентный спуск — двигатель обучения
- Тренировать глубокие сети сложно
- Но есть проверенные приёмы

Дальше:

- CNN и Computer Vision (lec03)
- RNN и последовательности (lec04)
- Attention и Transformer (lec06)
- Не забудем про практику!

«Богатство информации порождает бедность внимания»

— но Self-Attention эту проблему решает